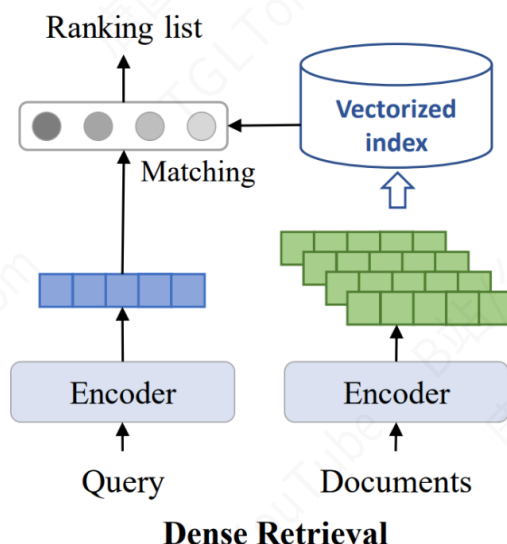


2.2 密集检索 DR, Dense Retriever

密集检索器（Dense Retriever, DR）是一种基于深度学习的信息检索技术，旨在通过理解文档和查询的深层语义信息来提高检索的准确性和效率。

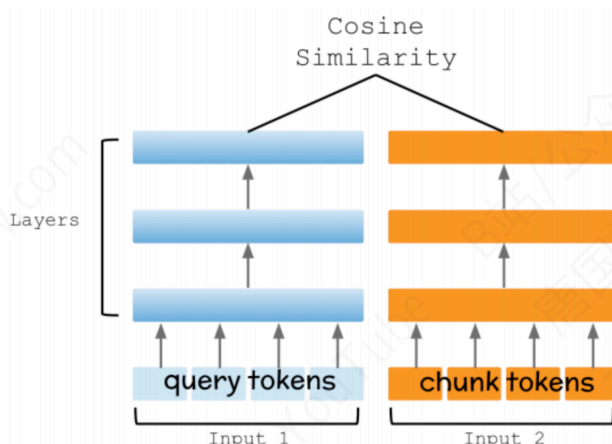
不同于传统的稀疏检索技术，如BM25，它依赖于显式的词项匹配，密集检索器利用预训练的神经网络模型，如BERT，来学习文档和查询的密集向量表示。这些向量表示能够捕捉到文本的深层语义信息，从而实现更精确的语义匹配。



2.2.1 DR的工作原理

1. 双编码器架构：DR通常采用双编码器架构，包括一个查询编码器和一个文档编码器。这两个编码器可以是同一个预训练模型的两个实例，也可以是不同的模型。查询编码器用于将用户查询转换为密集向量，而文档编码器将文档库中的每个文档转换为密集向量。

2. 向量相似度计算：一旦查询和文档被转换为向量，DR通过计算查询向量与文档向量之间的相似度来检索最相关的文档。相似度通常通过余弦相似性计算，选出相似度最高的前k个文档作为检索结果。



3. 预训练与微调：DR模型通常在大规模文本数据上进行预训练，以学习通用的语言表示。然后，可以通过在特定检索任务的标注数据上进行微调，来优化模型对该任务的性能。

2.2.2 DR的特点

(1) 优势

- 1. 语义匹配能力：**DR能够理解查询和文档的深层语义信息，即使没有直接的词项重叠，也能检索到语义上相关的文档。
- 2. 可扩展性：**尽管密集检索器需要在预训练阶段处理大量数据，但一旦模型被训练和优化，检索过程相对高效，特别是通过使用近似最近邻搜索（Approximate Nearest Neighbor, ANN）技术来加速向量搜索。
- 3. 适应性：**通过微调，DR可以适应不同的领域和任务，从而提供更定制化的检索服务。

(2) 劣势

- 1. 计算成本：**预训练和微调密集检索器需要大量的计算资源，特别是对于大规模文档库。
- 2. 冷启动问题：**对于新的检索任务，可能缺乏足够的标注数据来进行有效的微调。
- 3. 更新与维护：**文档库的更新可能要求重新计算文档向量，增加了系统的维护成本。

2.2.3 DR的应用

密集检索技术已经被广泛应用于各种信息检索任务中，包括但不限于：

- **问答系统：**通过理解问题和知识库文档的语义，精准检索到包含答案的文档。
- **文档检索：**在法律、医疗等领域提供基于语义的文档检索服务。
- **个性化推荐：**分析用户的查询和历史行为，检索出最相关的内容或商品。

>>> 举例讲解

(1) embedding 模型应用

密集检索案例huggingface.ipynb

embedding相似性计算的四种策略.ipynb

(2) embedding 模型微调

04_Embedding模型微调指南.md

(3) embedding 模型部署

05_embedding_model_deployment文件夹

文件-1: `embedding_app.py`

将embedding模型基于FastAPI封装为对外提供的API服务

文件-2: `embedding_test.py`

基于requests测试embedding API服务

文件-3: `embedding_concurrent_test.py`

评测API服务的平均响应时间, QPS (每秒查询率), 以下截图是测试结果:

成功请求次数: 1000 / 1000

总测试时间: 44.68秒

平均响应时间: 4.42秒

QPS (每秒查询率): 22.38

2.3 生成式检索 GR, Generative Retriever

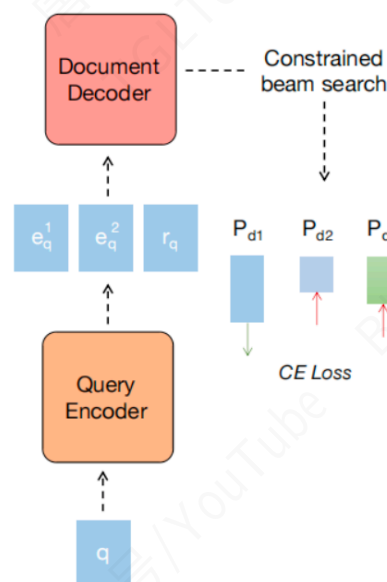
生成式检索 (Generative Retrieval, GR) 是一种信息检索技术, 它使用生成式模型来自动构造或生成与给定查询相关的文档标识符或文本响应。

生成式检索(Generative Retrieval) 的核心思想是构建一个 Seq2Seq 生成模型, 输入 query, 直接输出 docID。

在训练阶段, 生成模型将文档内容映射到语义唯一标志符(docID)实现文档的索引;

在推理阶段, 生成模型将查询映射到对应的文档 (docID) 以完成文档检索。

作为对比, 传统的双塔范式是索引+召回&重排。索引, 召回器, 重排器是可以互相独立训练的, 而 GR下面索引+召回&重排的过程直接融合在模型的训练和推理里面。



Memorizing
Generative Retrieval